



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ
ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

Λειτουργικά Συστήματα

Ακαδημαϊκό Έτος 2024-2025

ΤΕΧΝΙΚΗ ΑΝΑΦΟΡΑ 1^{ης} ΕΡΓΑΣΙΑΣ

ΚΙΟΥΡΤ-ΜΕΧΜΕΤΑΛΗ ΑΧΜΕΤ

A.M.: 1084672

up1084672@ac.upatras.gr

ΛΑΛΑΣ ΓΙΩΡΓΟΣ

A.M.: 1093406

up1093406@ac.upatras.gr

ΓΚΟΥΡΒΕΛΟΣ ΛΕΩΝΙΔΑΣ

A.M.: 1072506

up1072506@ac.upatras.gr

ΓΕΩΡΓΙΟΥ ΚΩΝΣΤΑΝΤΙΝΟΣ

A.M.: 1067419

up1067419@ac.upatras.gr

Ερώτημα 1

Shell Scripting

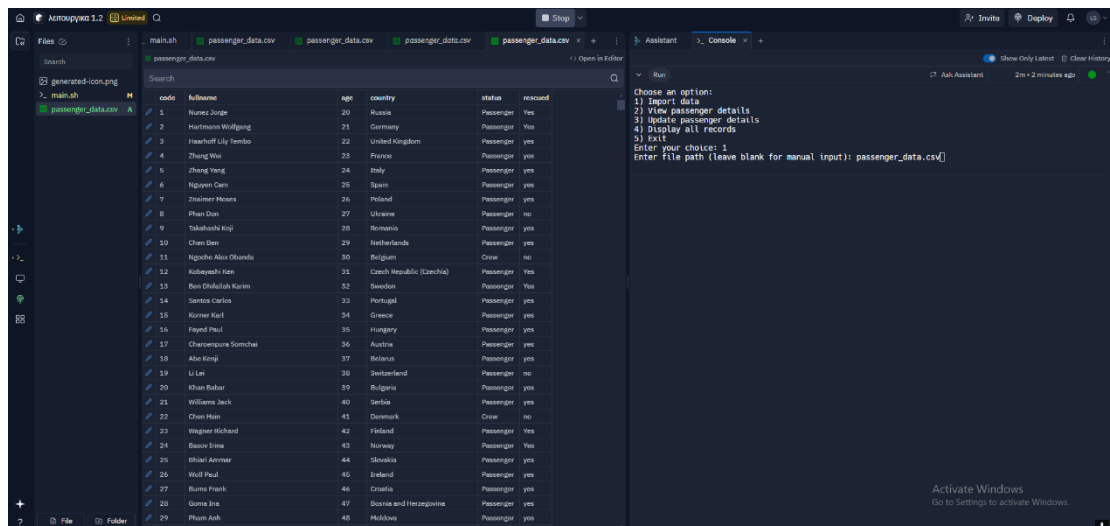
Το script λειτουργεί με την ακόλουθη λογική: Η συνάρτηση `display_menu` εμφανίζει τις διαθέσιμες επιλογές στον χρήστη και αποθηκεύει την επιλογή του σε μια μεταβλητή. Η συνάρτηση `import_data` δίνει τη δυνατότητα εισαγωγής δεδομένων με δύο τρόπους: μέσω ενός αρχείου CSV που καθορίζει ο χρήστης ή μέσω χειροκίνητης εισαγωγής δεδομένων (κωδικός, πλήρες όνομα, ηλικία, χώρα, κατάσταση, διάσωση). Η συνάρτηση `view_passenger` αναζητά έναν επιβάτη με βάση το όνομα και το επώνυμό του στο αρχείο `passengers.csv` και εμφανίζει τα αντίστοιχα αποτελέσματα. Η συνάρτηση `update_passenger` επιτρέπει την τροποποίηση των δεδομένων ενός επιβάτη, αφού πρώτα τον αναζητήσει είτε μέσω του μοναδικού κωδικού του είτε μέσω του ονόματος. Αν βρεθούν πολλαπλές εγγραφές, ο χρήστης καλείται να δώσει πιο ακριβή στοιχεία. Η ενημέρωση γίνεται μέσω της εντολής `sed` που αντικαθιστά την επιλεγμένη γραμμή στο αρχείο. Η συνάρτηση `display_all` χρησιμοποιεί την εντολή `less` για να εμφανίσει όλες τις εγγραφές του αρχείου `passengers.csv`. Ο χρήστης μπορεί να επιλέξει την έξοδο από το πρόγραμμα επιλέγοντας την αντίστοιχη επιλογή.

Το script εκτελείται σε έναν ατέρμονα βρόχο `while true`, εμφανίζοντας το μενού και διαχειριζόμενο τις επιλογές του χρήστη μέσω της εντολής `case`.

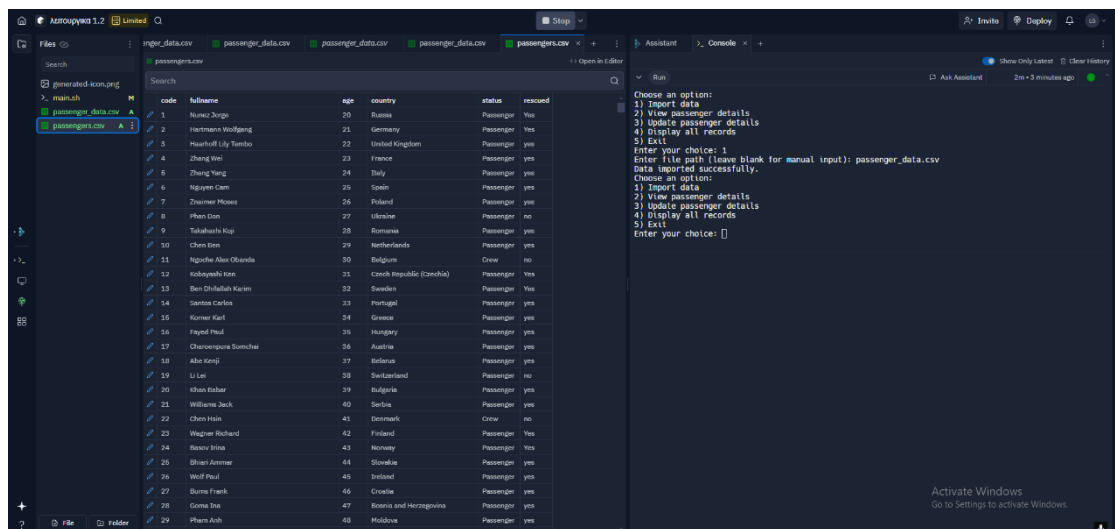
ΠΑΑΔΕΙΓΜΑΤΑ ΕΚΤΕΛΕΣΗΣ

Για την εκτέλεση χρησιμοποιήθηκε η online ιστοσελίδα `replit` για καλύτερη απεικόνιση.

ΛΕΙΤΟΥΡΓΙΑ 1:

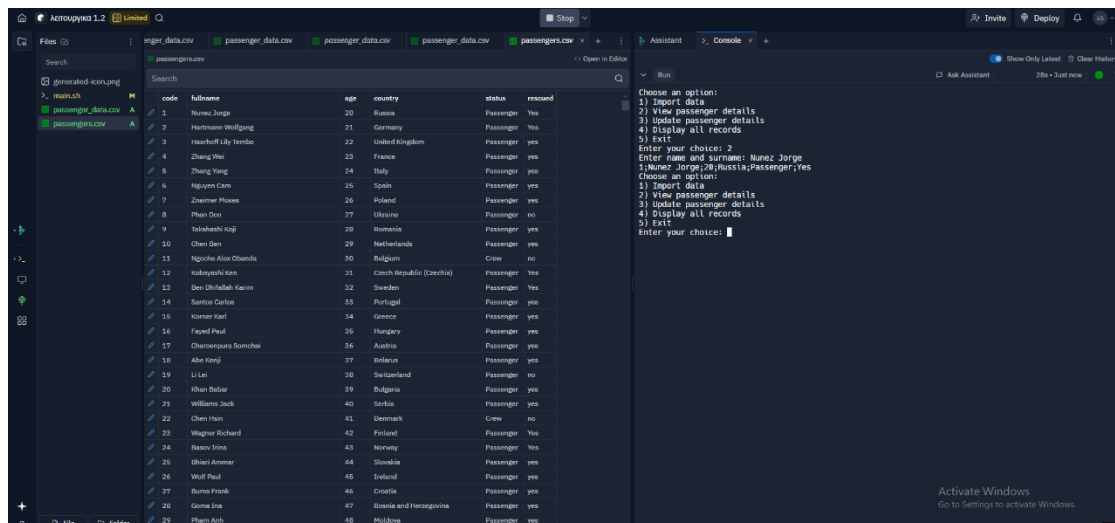


Στην παραπάνω εικόνα φαίνεται το αρχικό μενού, που υπάρχουν οι λειτουργίες. Έχουμε επιλέξει την λειτουργία 1 και μας ζητείται να εισάγουμε το file path στο οποίο πληκτρολογούμε το `passenger_data.csv`



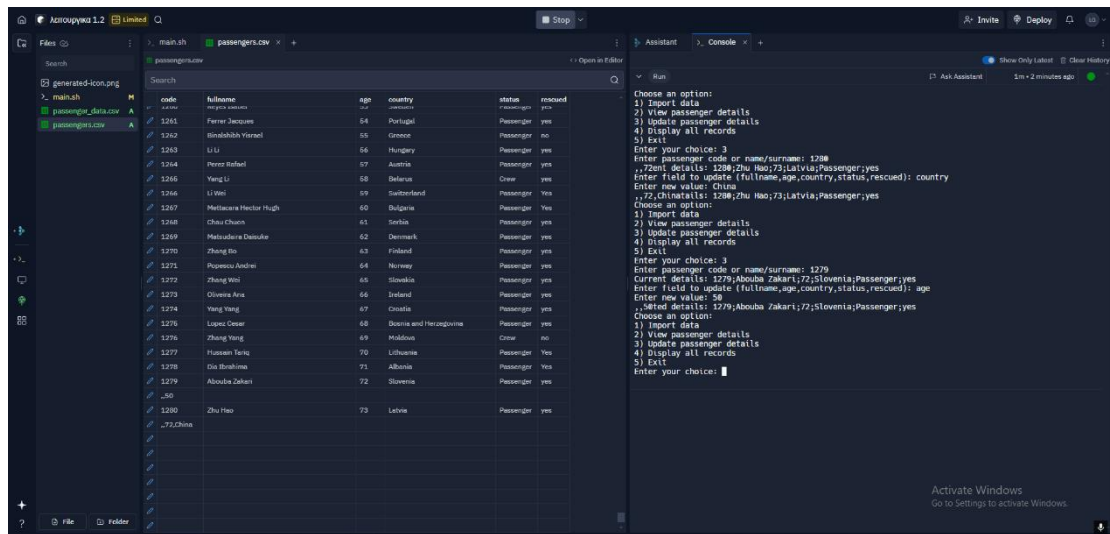
Όπως φαίνεται στην δεύτερη εικόνα τα δεδομένα του αρχείου έχουν εισαχθεί στο passenger.csv το οποίο είναι αυτό που χρησιμοποιείται στη main

ΛΕΙΤΟΥΡΓΙΑ 2:



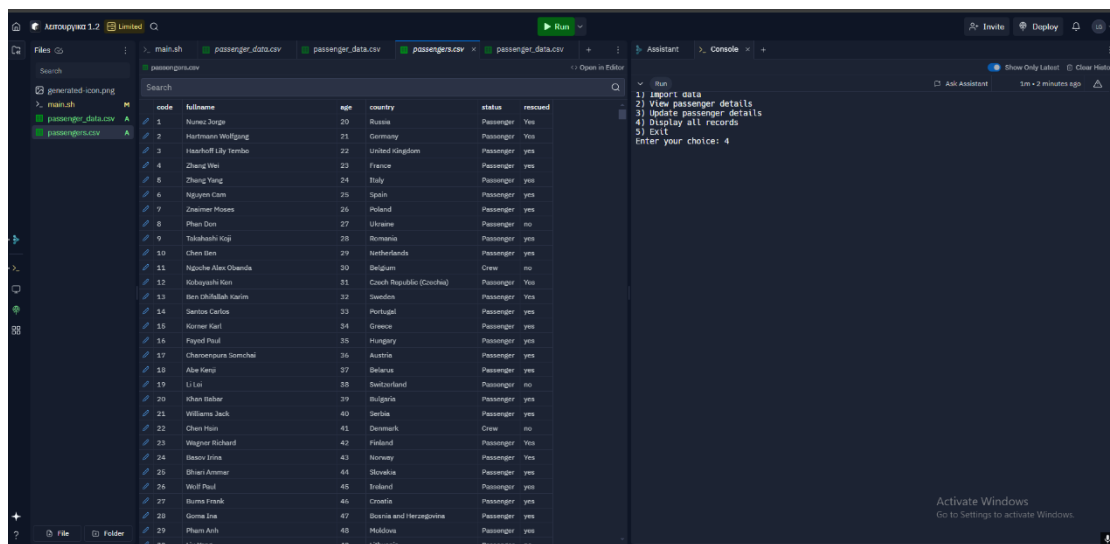
Στην παραπάνω εικόνα έχουμε επιλέξει τη λειτουργία 2 και μας ζητείται να το ονοματεπώνυμο του επιβάτη που θέλουμε να αναζητήσουμε. Για λόγους ευκολίας πληκτρολογούμε τον πρώτο επιβάτη με όνομα Nunez Jorge και μας εμφανίζει κανονικά τα υπόλοιπα του στοιχεία ακριβώς έτσι όπως είναι γραμμένα στο αρχείο. Η λειτουργία στην παρούσα υλοποίηση λειτουργεί με όρισμα ολόκληρο το ονοματεπώνυμο του επιβάτη και όχι μόνο με το όνομα ή το επώνυμο.

ΛΕΙΤΟΥΡΓΙΑ 3:



Στην τρίτη λειτουργία ενώ επιλέγει τον σωστό επιβάτη, όπως φαίνεται στην εικόνα δεν λειτουργεί σωστά καθώς αντί να κάνει ενημέρωση του πίνακα προσθέτει νέα στοιχεία στον επιβάτη που έχει επιλεχτεί.

ΛΕΙΤΟΥΡΓΙΑ 4:



code	full name	age	country	status	rescued
1	Nunez Jorge	20	Russia	Passenger	Yes
2	Hartmann Wolfgang	21	Germany	Passenger	Yes
3	Haarhoff Lily Tembo	22	United Kingdom	Passenger	Yes
4	Zhang Wei	23	France	Passenger	Yes
5	Zhang Yang	24	Italy	Passenger	Yes
6	Nguyen Cam	25	Spain	Passenger	Yes
7	Zimmer Moses	26	Poland	Passenger	Yes
8	Phan Don	27	Ukraine	Passenger	No
9	Takahashi Koji	28	Kosovo	Passenger	Yes
10	Chen Ben	29	Netherlands	Passenger	Yes
11	Nguyen Alex Obando	30	Belgium	Crew	No
12	Kobayashi Ken	31	Czech Republic (Czechia)	Passenger	Yes
13	Ben Othallah Karon	32	Sweden	Passenger	Yes
14	Santos Carlos	33	Portugal	Passenger	Yes
15	Korner Karl	34	Greece	Passenger	Yes
16	Fayed Paul	35	Hungary	Passenger	Yes
17	Charoenpura Somchai	36	Austria	Passenger	Yes
18	Ali Kenji	37	Belarus	Passenger	Yes
19	Li Lei	38	Switzerland	Passenger	No
20	Khan Behar	39	Bulgaria	Passenger	Yes
21	Williams Jack	40	Serbia	Passenger	Yes
22	Chen Hsin	41	Denmark	Crew	No
23	Wagner Richard	42	Finland	Passenger	Yes
24	Basov Irina	43	Norway	Passenger	Yes
25	Bhati Ammar	44	Slovakia	Passenger	Yes
26	Wolf Paul	45	Ireland	Passenger	Yes
27	Burns Frank	46	Croatia	Passenger	Yes
28	Goma Ina	47	Bosnia and Herzegovina	Passenger	Yes
29	Pham Anh	48	Moldova	Passenger	Yes

Όπως φαίνεται από τον φάκελο, έχουν τυπωθεί σωστά τα στοιχεία του πίνακα.

Ερώτημα 2

Συγχρονισμός Διεργασιών και Σημαφόροι

Για την συγκεκριμένη άσκηση έχουμε δημιουργήσει τους κώδικες που μπορείτε να παρατηρήσετε και από το .zip αρχείο που έχουμε τους κώδικες.

Στην ουσία έχουμε lifeboats που περιλαμβάνουν κάποιο όριο για τους επιβάτες ώστε να μπορούν να σωθούν που ορίζουμε και το αριθμό των επιβατών για οποίους θα ανεβούν στο lifeboat.

Επίσης έχουμε τα σενάρια που ο επιβάτης μπορεί να αλλάξει γνώμη για την απόφαση το αν θέλει να ανεβεί στο lifeboat ή όχι.

Δυστυχώς, δεν έχουμε την λειτουργία του κώδικα όπως θα πρέπει, δηλαδή τα lifeboats να μπαίνουν σε κάποιο queue ώστε να μπορούν παραπάνω επιβάτες να σωθούν. Επίσης, ανεξαρτήτως το capacity του lifeboat μπορεί μόνο ένας επιβάτης να ανεβεί στο lifeboat.

Σενάρια

```
kmachmet@achmet-virtualbox:~/my_project$ dir
ipc_utils.c ipc_utils.h ipc_utils.h.save launch.c my_program passenger.c
kmachmet@achmet-virtualbox:~/my_project$ gcc -o my_program ipc_utils.c passenger.c launch.c -lpthread
kmachmet@achmet-virtualbox:~/my_project$ ./my_program
Enter the total number of passengers: 3
Enter the lifeboat capacity: 3
Passenger 1: Waiting to board the lifeboat...
Passenger 1: Boarding the lifeboat.
Passenger 2: Waiting to board the lifeboat...
Passenger 3: Waiting to board the lifeboat...
Passenger 1: Successfully boarded.
Passenger 2: Boarding the lifeboat.
Passenger 2: Successfully boarded.
Passenger 3: Boarding the lifeboat.
Passenger 3: Successfully boarded.
All passengers have boarded. Lifeboat process complete.
kmachmet@achmet-virtualbox:~/my_project$
```

```
kmachmet@achmet-virtualbox:~/my_project$ ./my_program
Enter the total number of passengers: 4
Enter the lifeboat capacity: 10
Passenger 1: Waiting to board the lifeboat...
Passenger 1: Boarding the lifeboat.
Passenger 2: Waiting to board the lifeboat...
Passenger 3: Waiting to board the lifeboat...
Passenger 4: Waiting to board the lifeboat...
Passenger 1: Successfully boarded.
Passenger 2: Boarding the lifeboat.
Passenger 2: Successfully boarded.
Passenger 3: Boarding the lifeboat.
Passenger 3: Successfully boarded.
Passenger 4: Changed their mind! Leaving the queue and rejoining at the end.
Passenger 4: Waiting to board the lifeboat...
Passenger 4: Boarding the lifeboat.
Passenger 4: Successfully boarded.
All passengers have boarded. Lifeboat process complete.
kmachmet@achmet-virtualbox:~/my_project$
```

Κώδικες

passenger.c

Ο κώδικας παρουσιάζει ένα σενάριο όπου επιβάτες προσπαθούν να επιβιβαστούν σε μια σωστική λέμβο με περιορισμένη χωρητικότητα. Χρησιμοποιούνται `semaphores` και `mutexes` για τη διαχείριση της σειράς αναμονής και της πρόσβασης στη λέμβο, ενώ παράλληλα υπάρχει πιθανότητα οι επιβάτες να αλλάξουν γνώμη.

Η λειτουργία `passenger_board` είναι η κύρια λογική κάθε επιβάτη. Κάθε επιβάτης προσπαθεί να αποκτήσει πρόσβαση στη σειρά αναμονής μέσω ενός `semaphore`. Αν υπάρχει διαθέσιμος χώρος στη λέμβο, ελέγχεται αν ο επιβάτης θα αλλάξει γνώμη (με πιθανότητα 20%). Αν αλλάξει γνώμη, απελευθερώνει τον χώρο και επανέρχεται στη σειρά. Διαφορετικά, επιβιβάζεται και τερματίζει τη διαδικασία.

launch.c

Ο κώδικας υλοποιεί τη διαδικασία επιβίβασης επιβατών σε σωστική λέμβο μέσω πολυνηματικής επεξεργασίας (`multithreading`) και χρήσης σημάτων (`semaphores`) για τον συγχρονισμό. Περιλαμβάνει τη δυνατότητα διαχείρισης ενός μέγιστου αριθμού επιβατών και τον καθορισμό της χωρητικότητας της σωστικής λέμβου από τον χρήστη. Αρχικά, ο χρήστης εισάγει τον συνολικό αριθμό επιβατών και τη χωρητικότητα της σωστικής λέμβου. Εάν ο αριθμός των επιβατών υπερβεί το καθορισμένο μέγιστο όριο, το πρόγραμμα τερματίζει με μήνυμα σφάλματος.

Ο κώδικας χρησιμοποιεί δύο σήματα: το `lifeboat_capacity` για να περιορίσει τον αριθμό των επιβατών που μπορούν να επιβιβαστούν ταυτόχρονα, και το `queue_access` για να διασφαλίσει την ομαλή πρόσβαση στη σειρά αναμονής. Αυτά τα σήματα αρχικοποιούνται μέσω των συναρτήσεων `init_semaphore` που υλοποιούνται εξωτερικά (στο αρχείο `ipc_utils.h`).

Για κάθε επιβάτη, δημιουργείται ένα ξεχωριστό νήμα που αναλαμβάνει την προσομοίωση της επιβίβασής του, μέσω της συνάρτησης `passenger_board`, η οποία υλοποιείται σε άλλο αρχείο (`passenger.c`).

Τα νήματα εκτελούνται παράλληλα, ενώ εφαρμόζεται μια μικρή καθυστέρηση για την προσομοίωση της ροής επιβίβασης. Μετά την ολοκλήρωση όλων των νημάτων, το πρόγραμμα περιμένει την ολοκλήρωσή τους (`pthread_join`) πριν καταστρέψει τα σήματα και τερματίσει.

ipc_utils.c

Ο κώδικας παρέχει μια σειρά από βοηθητικές συναρτήσεις για τη διαχείριση των `semaphores` που χρησιμοποιούνται για τον συγχρονισμό σε προγράμματα πολυνηματικής επεξεργασίας. Οι συναρτήσεις αυτές υλοποιούνται στο αρχείο `ipc_utils.h` και είναι σχεδιασμένες για να απλοποιούν τη χρήση των σημάτων στη ροή του προγράμματος.

Η συνάρτηση `init_semaphore` αρχικοποιεί ένα σήμα με μια συγκεκριμένη αρχική τιμή, ενώ η `destroy_semaphore` χρησιμοποιείται για την αποδέσμευση των πόρων που σχετίζονται με το σήμα μετά τη χρήση.

Οι συναρτήσεις `wait_semaphore` και `signal_semaphore` εφαρμόζουν τις βασικές λειτουργίες `wait` και απελευθέρωσης `signal` σε ένα σήμα, επιτρέποντας τον έλεγχο της πρόσβασης σε κοινόχρηστους πόρους από πολλαπλά νήματα.

`ipc_utils.h`

Ο κώδικας είναι αρχείο header file που ορίζει λειτουργίες και μεταβλητές για τη διαχείριση των `semaphores` και `threads`. Ο σκοπός του είναι να παρέχει ένα κοινό σημείο αναφοράς για τη χρήση σημάτων σε προγράμματα πολυνηματικής επεξεργασίας.

Περιλαμβάνει δηλώσεις για τέσσερις συναρτήσεις που σχετίζονται με τα σήματα: `init_semaphore`, `wait_semaphore`, `signal_semaphore` και `destroy_semaphore`. Αυτές οι συναρτήσεις υλοποιούνται σε άλλο αρχείο πηγαίου κώδικα και διευκολύνουν τη χρήση σημάτων στον συγχρονισμό.

Επίσης, ορίζει δύο κοινόχρηστα σήματα, το `lifeboat_capacity` και το `queue_access`, τα οποία δηλώνονται ως `extern` για να είναι ορατά σε άλλα αρχεία που περιλαμβάνουν το παρόν. Αυτά χρησιμοποιούνται για τον έλεγχο της πρόσβασης σε κοινόχρηστους πόρους. Το αρχείο αυτό εξασφαλίζει ότι οι συναρτήσεις και οι μεταβλητές μπορούν να χρησιμοποιηθούν εύκολα σε πολλαπλά αρχεία κώδικα, προσφέροντας έτσι οργανωμένη και επαναχρησιμοποιήσιμη λειτουργικότητα.

Ερώτημα 3

Χρονοπρογραμματισμός Διεργασιών και Διαχείριση Μνήμης

Θα πρέπει να υλοποιήσουμε έναν κώδικα Round Robin όπου θα εκτελεί τον αλγόριθμο ανάλογα με τα pid, arrival_time, remaining_time και memory needed.

Ο κώδικας περιλαμβάνει δύο κύριες δομές. Η δομή Process αποθηκεύει πληροφορίες για κάθε διεργασία, όπως ο χρόνος άφιξης, η διάρκεια και η μνήμη που απαιτεί. Η δομή MemoryBlock αναπαριστά τμήματα της μνήμης, περιλαμβάνοντας το μέγεθος, την κατάσταση (ελεύθερο ή κατειλημμένο) και το PID της διεργασίας που το χρησιμοποιεί.

Η μνήμη αρχικοποιείται ως ένα μεγάλο μπλοκ και διαχειρίζεται με τον αλγόριθμο First Fit. Όταν μια διεργασία ζητά μνήμη, ελέγχεται το πρώτο διαθέσιμο τμήμα που επαρκεί. Εάν περισσεύει χώρος, δημιουργείται νέο μπλοκ. Όταν ολοκληρώνεται η διεργασία, η μνήμη αποδεδεσμεύεται και, αν είναι δυνατό, συνενώνονται γειτονικά ελεύθερα μπλοκ.

Ο προγραμματισμός των διεργασιών γίνεται με τον αλγόριθμο Round-Robin. Ο χρήστης εισάγει τα δεδομένα των διεργασιών, και αυτές εκτελούνται για ένα προκαθορισμένο χρονικό διάστημα (Time Quantum). Εάν μια διεργασία δεν μπορεί να εισέλθει στη μνήμη λόγω έλλειψης χώρου, παραμένει σε αναμονή. Κατά την εκτέλεση, ο κώδικας εμφανίζει την κατάσταση της μνήμης και των διεργασιών, παρέχοντας σαφή εικόνα της λειτουργίας.

Ο κώδικας υλοποιεί τη χρονοδρομολόγηση διεργασιών και τη διαχείριση μνήμης για ένα σύστημα με 512 KB μνήμη. Οι διεργασίες προγραμματίζονται με τον αλγόριθμο **Round Robin** (χρονικό κβάντο 3 ms), ενώ η μνήμη κατανέμεται με τον αλγόριθμο **First Fit**. Η μνήμη προσομοιώνεται ως πίνακας μπλοκ, όπου κάθε μπλοκ περιέχει πληροφορίες για τη διεύθυνση, το μέγεθος, την κατάστασή του (ελεύθερο ή κατειλημμένο) και το PID της διεργασίας. Κατά την κατανομή, αν ένα μπλοκ είναι μεγαλύτερο από το απαιτούμενο, χωρίζεται σε δύο μέρη. Όταν μια διεργασία ολοκληρωθεί, η μνήμη της απελευθερώνεται, και αν γειτονικά μπλοκ είναι ελεύθερα, ενώνονται.

Η χρονοδρομολόγηση των διεργασιών γίνεται κυκλικά. Οι διεργασίες που έχουν φτάσει και διαθέτουν μνήμη εκτελούνται για 3 ms ή για τον υπολειπόμενο χρόνο τους. Αν δεν υπάρχει διαθέσιμη μνήμη, περιμένουν. Μετά από κάθε κύκλο, εμφανίζεται η κατάσταση της μνήμης και των διεργασιών. Το σύστημα ολοκληρώνεται όταν όλες οι διεργασίες έχουν εκτελεστεί και η μνήμη έχει απελευθερωθεί πλήρως.

Ο κώδικας που έχουμε δημιουργήσει υλοποιεί τον αλγόριθμο Round Robin όπου θα πρέπει να ορίσουμε τα commands όπως θα μπορούσαμε να παρατηρήσουμε και στα σενάρια που θα έχουμε.

Σενάρια:

```
Enter the number of processes: 3
Enter details for process 1 (arrival_time duration memory_needed): 0 2 100
Enter details for process 2 (arrival_time duration memory_needed): 2 2 100
Enter details for process 3 (arrival_time duration memory_needed): 4 2 100
Time 0: Process 1 allocated memory.
Time 0: Process 1 executing for 2 ms.
Time 2: Process 1 completed.
Time 2: Process 2 allocated memory.
Time 2: Process 2 executing for 2 ms.
Time 4: Process 2 completed.
Time 4: Process 3 allocated memory.
Time 4: Process 3 executing for 2 ms.
Time 6: Process 3 completed.
Memory:
Block 0: Start=0, Size=512, Free=Yes, PID=-1

Processes:
PID=1, Arrival=0, Duration=2, Remaining=0, MemoryNeeded=100, InMemory=No
PID=2, Arrival=2, Duration=2, Remaining=0, MemoryNeeded=100, InMemory=No
PID=3, Arrival=4, Duration=2, Remaining=0, MemoryNeeded=100, InMemory=No

All processes completed.

Process returned 0 (0x0)   execution time : 19.792 s
Press any key to continue.
```

Για το πρώτο σενάριο όπως μπορείτε να παρατηρήσετε έχουμε τις διεργασίες με τα commands που χρειάζεται για την εκτέλεση ώστε να μπορεί να υλοποιηθεί ο αλγόριθμος με την σωστή σειρά αλλά και σωστή εκτέλεση.

```

Enter the number of processes: 5
Enter details for process 1 (arrival_time duration memory_needed): 0 5 100
Enter details for process 2 (arrival_time duration memory_needed): 1 3 100
Enter details for process 3 (arrival_time duration memory_needed): 3 6 100
Enter details for process 4 (arrival_time duration memory_needed): 5 1 100
Enter details for process 5 (arrival_time duration memory_needed): 6 4 100
Time 0: Process 1 allocated memory.
Time 0: Process 1 executing for 3 ms.
Time 3: Process 2 allocated memory.
Time 3: Process 2 executing for 3 ms.
Time 6: Process 2 completed.
Time 6: Process 3 allocated memory.
Time 6: Process 3 executing for 3 ms.
Time 9: Process 4 allocated memory.
Time 9: Process 4 executing for 1 ms.
Time 10: Process 4 completed.
Time 10: Process 5 allocated memory.
Time 10: Process 5 executing for 3 ms.
Memory:
Block 0: Start=0, Size=100, Free=No, PID=1
Block 1: Start=100, Size=100, Free=No, PID=3
Block 2: Start=200, Size=100, Free=No, PID=5
Block 3: Start=300, Size=212, Free=Yes, PID=-1

Processes:
PID=1, Arrival=0, Duration=5, Remaining=2, MemoryNeeded=100, InMemory=Yes
PID=2, Arrival=1, Duration=3, Remaining=0, MemoryNeeded=100, InMemory=No
PID=3, Arrival=3, Duration=6, Remaining=3, MemoryNeeded=100, InMemory=Yes
PID=4, Arrival=5, Duration=1, Remaining=0, MemoryNeeded=100, InMemory=No
PID=5, Arrival=6, Duration=4, Remaining=1, MemoryNeeded=100, InMemory=Yes

Time 13: Process 1 executing for 2 ms.
Time 15: Process 1 completed.
Time 15: Process 3 executing for 3 ms.
Time 18: Process 3 completed.
Time 18: Process 5 executing for 1 ms.
Time 19: Process 5 completed.
Memory:
Block 0: Start=0, Size=512, Free=Yes, PID=-1

Processes:
PID=1, Arrival=0, Duration=5, Remaining=0, MemoryNeeded=100, InMemory=No
PID=2, Arrival=1, Duration=3, Remaining=0, MemoryNeeded=100, InMemory=No
PID=3, Arrival=3, Duration=6, Remaining=0, MemoryNeeded=100, InMemory=No
PID=4, Arrival=5, Duration=1, Remaining=0, MemoryNeeded=100, InMemory=No
PID=5, Arrival=6, Duration=4, Remaining=0, MemoryNeeded=100, InMemory=No

All processes completed.

Process returned 0 (0x0) execution time : 26.402 s
Press any key to continue.

```

Άλλη μία περίπτωση που μπορείτε να παρατηρήσετε και στην παραπάνω εικόνα είναι ότι ο αλγόριθμος εκτελείτε στο σωστό χρονικό διάστημα χωρίς κάποια καθυστέρηση, όμως δεν εκτελείτε με τον τρόπο που θα χρειάζεται. Παρόλο που η εκτέλεση ξεκινάει όπως θα έπρεπε στην συνέχεια η τοποθέτηση των διεργασιών για εκτέλεση γίνεται με σφάλματα.

Ερώτημα 4

Χρονοπρογραμματισμός Διεργασιών

α)

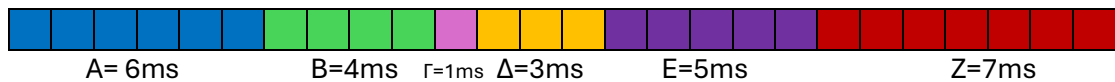
- **FCFS (First Come First Served)**

Στον αλγόριθμο FCFS υλοποιείται κάθε φορά η διεργασία που βρίσκεται πρώτη στην σειρά. Για αυτόν τον λόγο το διάγραμμα Gantt είναι αρκετά απλό καθώς οι διεργασίες υλοποιούνται απλά με την σειρά.

Τα χρώματα που έχει η κάθε διεργασία

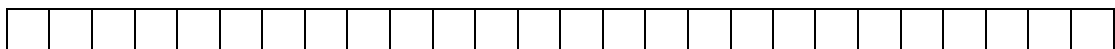


Συνολικά 26ms



- **SJF (Shortest Job First)**

Στον αλγόριθμο SJF υλοποιείται κάθε φορά η διεργασία που έχει τον μικρότερο χρόνο εκτέλεσης. Ξεκινώντας, την χρονική στιγμή 0 φτάνει η διεργασία Α.

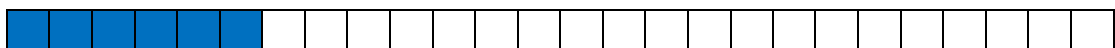


A=6

Χρόνος Άφιξης=0

Έτσι την προσθέτουμε πρώτη στο διάγραμμα:

A=6ms



Την χρονική στιγμή 2 φτάνει η διεργασία B=4,

Την χρονική στιγμή 3 φτάνει η διεργασία Γ=1,

Την χρονική στιγμή 4 φτάνει η διεργασία Δ=3,

Την χρονική στιγμή 5 φτάνει η διεργασία Ε=5 και

Την χρονική στιγμή 6 φτάνει η διεργασία Ζ=7.

Αφού έχουν φτάσει όλες οι διεργασίες προσθέτουμε κάθε διεργασία με αύξουσα σειρά του χρόνου εκτέλεσης.

Συγκρίνοντας τες συμπεραίνουμε ότι αυτή με τον μικρότερο χρόνο εκτέλεσης είναι η Γ. Έτσι την προσθέτουμε στο διάγραμμα:

A=6ms

Γ=1ms



Έπειτα, συνεχίζουμε με την ίδια λογική και στις άλλες διεργασίες:

A=6ms

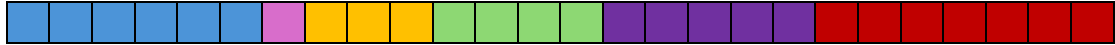
Γ=1ms

Δ=3ms

B=4ms

E=5ms

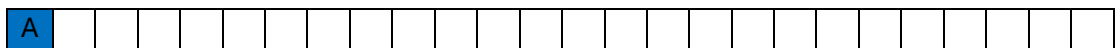
Z=7ms



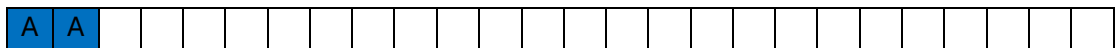
- **SRTF (Shortest Remaining Time First)**

Στον αλγόριθμο SRTF οι διεργασίες υλοποιούνται με βάση τον εναπομείναντα χρόνο εκτέλεσης. Ξεκινώντας, την χρονική στιγμή 0 φτάνει η διεργασία A.

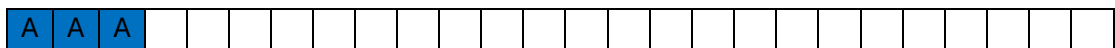
Που την προσθέτουμε και $A=6-1=5$:



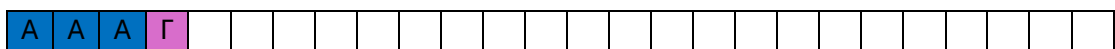
Έπειτα, αφού δεν έχει έρθει κάποια άλλη διεργασία, και δεν υπάρχει άλλη διεργασία για σύγκριση συνεχίζουμε και βάζουμε ξανά το A ώστε $A=5-1=4$.



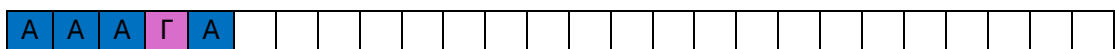
Στην χρονική στιγμή 2 έχει φτάσει και η διεργασία B, παρατηρούμε όμως ότι έχει μεγαλύτερο εναπομείναντα χρόνο εκτέλεσης. Άρα συνεχίζουμε με την διεργασία A και έχουμε $4-1=3$.



Στην χρονική στιγμή 3 φτάνει και η διεργασία Γ, που έχει λιγότερο εναπομείναντα χρόνο εκτέλεσης από την A και B. Με αποτέλεσμα αυτή να προστίθεται στην ουρά $1-1=0$.



Βρισκόμαστε στην χρονική στιγμή 4 φτάνει και η διεργασία Δ που περιέχει 3 σαν εναπομείναντα χρόνο εκτέλεσης. Σε αυτή την κατάσταση έχουμε ισότητα της διεργασία A με την διεργασία Δ. Επιλέγουμε αυτήν που είχε μικρότερο χρόνο άφιξης και προσθέτουμε το A ξανά στην ουρά με αποτέλεσμα να μειώνεται και πάλι η διάρκεια εκτέλεσης σε $A=3-1=2$.



Στην χρονική στιγμή 5 φτάνει η διεργασία E, που έχει 5 σαν εναπομείναντα χρόνο εκτέλεσης. Που σε αυτή την φάση για εναπομείναντα χρόνο εκτέλεσης έχουμε:

$A=2$

B=4

Γ=0 (έχει ήδη λήξει)

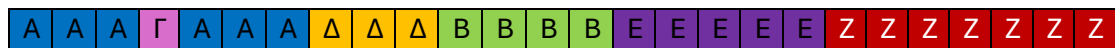
Δ=3

E=5

Που σημαίνει ότι θα συνεχίσουμε με την A με αποτέλεσμα να έχουμε $A=2-1=1$



Τώρα που βρισκόμαστε στην χρονική στιγμή 6 φτάνει και η τελευταία διεργασία όπου $Z=7$. Έχουν φτάσει όλες οι διεργασίες που σημαίνει ότι ο αλγόριθμος μεταβάλλεται σε μορφή όπου θα δουλέψει σε SJF αλγόριθμό όπου βάζουμε την κάθε διεργασία με όλα τα εναπομείναντα χρόνο εκτέλεσης που έχουν. Βάζουμε την A πρώτη όπου έχει μόνο 1 εναπομείναντα χρόνο εκτέλεσης και στην συνέχεια προσθέτουμε και τις άλλες διεργασίες.

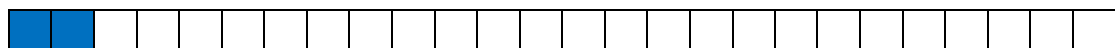


• RR (Round Robin) με κβάντο χρόνου 2 χρονικές μονάδες

Στον αλγόριθμο Round Robin η κάθε διεργασία που επιλέγεται για εκτέλεση μπορεί να εκτελεστεί μόνο για 2 χρονικές μονάδες. Έπειτα πηγαίνει τελευταία στην ουρά προτεραιότητας και ξαναυλοποιείται για 2 χρονικές μονάδες όταν ξαναέρθει η σειρά της.

Όταν 2 διεργασίες φτάνουν ταυτόχρονα χρησιμοποιούμε τον FCFS για να επιλέξουμε ποια θα εκτελεστεί πρώτη.

1. Ξεκινώντας, την χρονική στιγμή 0 φτάνει η διεργασία A εκτελείται για 2 ms.



A=2ms

2. Μετά την εκτέλεση των 2ms για την A, την χρονική στιγμή 2 φτάνει η διεργασία B και προστίθεται στην ουρά, που μετά το B, βάζουμε και το A στην ουρά αφού δεν έχει τελειώσει όλοι η εκτέλεση $A=6-2=4$.

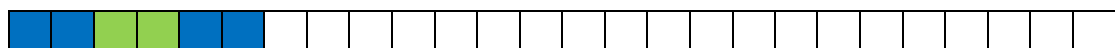


A=2ms B=2ms

Που μετά την εκτέλεση αυτή έχουμε για την B $B=4-2=2$

Την χρονική στιγμή 3 και 4 φτάνουν οι διεργασίες Γ και Δ αντίστοιχα, που θα προστεθούν στην ουρά πίσω από την A. Πίσω από την Γ και Δ προσθέτουμε την B αφού δεν έχει τελειώσει όλη η εκτέλεση. Άρα προς το παρόν η ουρά είναι **A B A Γ Δ B**.

3. Εκτελούμε την A και πάλι για 2ms:



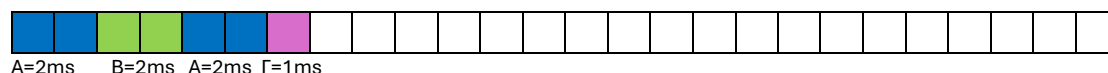
A=2ms B=2ms A=2ms

Την χρονική στιγμή 5 και 6 φτάνουν οι διεργασίες Ε και Ζ. Που θα προστεθούν στην ουρά. Στην συνέχεια, αφού η $A=4-2=2$ έχει ακόμα κομμάτια εκτέλεσης θα πάει από πίσω των Ε και Ζ.

Η ουρά που έχουμε είναι: **A B A** Γ Δ Β Ε Ζ Α.

Σε αυτή την φάση έχουν φτάσει όλες οι διεργασίες που σημαίνει ότι από εδώ και πέρα θα προστεθούν στην ουρά οι μόνοι οι διεργασίες που έχουν ακόμα κομμάτια εκτέλεσης.

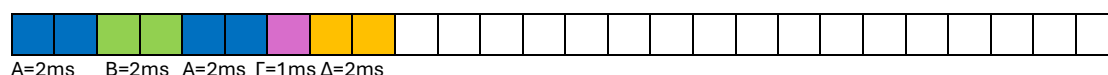
4. Εκτελούμε την Γ:



Επειδή λήγει η $\Gamma=1-1=0$ και δεν έχει άλλα κομμάτια εκτέλεσης δεν προστίθεται στην ουρά.

Η ουρά που έχουμε: **A B A F** Δ Β Ε Ζ Α.

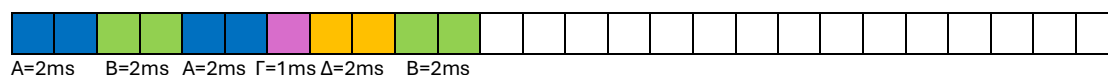
5. Εκτελούμε την Δ:



Που σημαίνει ότι $\Delta=3-2=1$ και προστίθεται στην ουρά.

Η ουρά που έχουμε: **A B A F Δ** Β Ε Ζ Α Δ.

6. Εκτελούμε την Β:



Που σημαίνει $B=2-2=0$.

Αφού έχει λήξει η Β έχουμε την ουρά που δεν προστίθεται η Β.

Η ουρά που έχουμε: **A B A F Δ B** Ε Ζ Α Δ.

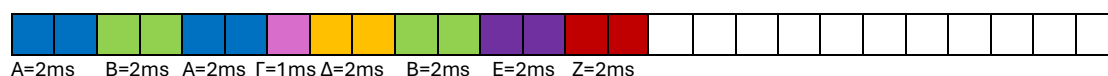
7. Εκτελούμε την Ε:



Που σημαίνει $E=5-2=3$ και προστίθεται στην ουρά για την εκτέλεση υπόλοιπου διάρκειας.

Η ουρά που έχουμε: **A B A F Δ B E** Ζ Α Δ Ε.

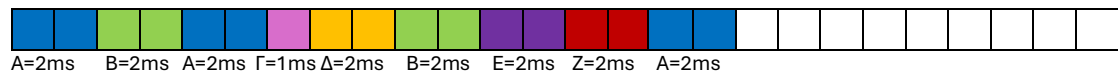
8. Εκτελούμε την Ζ:



Που σημαίνει $Z=7-2=5$.

Η ουρά που έχουμε: **A B A F Δ B E Z** Α Δ Ε Ζ.

9. Εκτελούμε την A:



Που σημαίνει $A=2-2=0$

Η ουρά που έχουμε: ~~A~~ ~~B~~ ~~A~~ ~~F~~ ~~Δ~~ ~~B~~ ~~E~~ ~~Z~~ ~~A~~ Δ E Z.

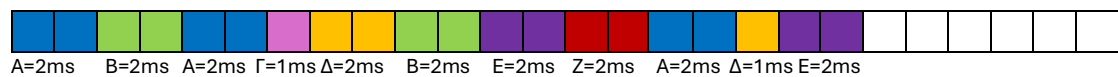
10. Εκτελούμε την Δ:



Που σημαίνει $Δ=1-1=0$

Η ουρά που έχουμε: ~~A~~ ~~B~~ ~~A~~ ~~F~~ ~~Δ~~ ~~B~~ ~~E~~ ~~Z~~ ~~A~~ ~~Δ~~ E Z.

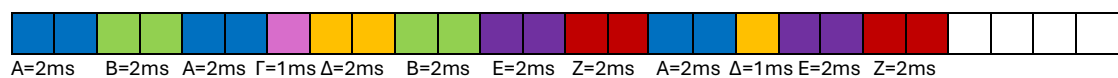
11. Εκτελούμε την E:



Που σημαίνει $E=3-2=1$

Η ουρά που έχουμε: ~~A~~ ~~B~~ ~~A~~ ~~F~~ ~~Δ~~ ~~B~~ ~~E~~ ~~Z~~ ~~A~~ ~~Δ~~ ~~E~~ Z E.

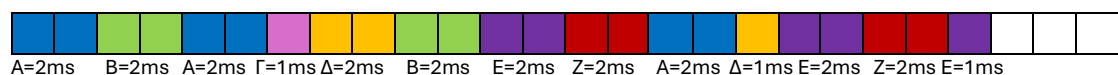
12. Εκτελούμε την Z:



Που σημαίνει $Z=5-2=3$.

Η ουρά που έχουμε: ~~A~~ ~~B~~ ~~A~~ ~~F~~ ~~Δ~~ ~~B~~ ~~E~~ ~~Z~~ ~~A~~ ~~Δ~~ ~~E~~ ~~Z~~ E Z.

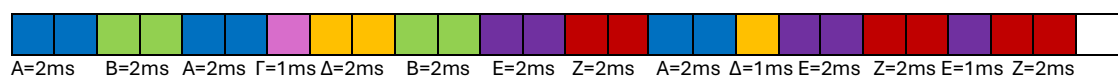
13. Εκτελούμε την E:



Που σημαίνει $E=1-1=0$

Η ουρά που έχουμε ~~A~~ ~~B~~ ~~A~~ ~~F~~ ~~Δ~~ ~~B~~ ~~E~~ ~~Z~~ ~~A~~ ~~Δ~~ ~~E~~ ~~Z~~ E Z.

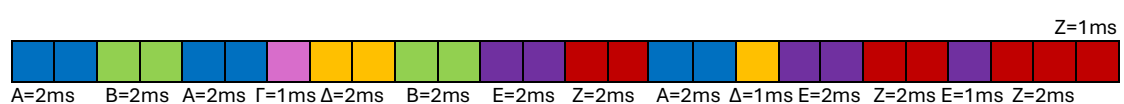
14. Εκτελούμε την Z:



Που σημαίνει $Z=3-2=1$.

Η ουρά που έχουμε ~~A~~ ~~B~~ ~~A~~ ~~F~~ ~~Δ~~ ~~B~~ ~~E~~ ~~Z~~ ~~A~~ ~~Δ~~ ~~E~~ ~~Z~~ E Z.

15. Εκτελούμε την Z:



Που σημαίνει $Z=1-1=0$

Η ουρά τελείωσε και έχουμε: ~~A B A F Δ B E Z A Δ E Z E Z Z~~

β) Για τα παρακάτω έστω ότι:

t_1 = χρονική στιγμή εισόδου,

MXA = Μέσος Χρόνος Αναμονής,

t_2 = χρονική στιγμή εξόδου,

t_{ap} = χρόνος απόκρισης,

t_{cpu} = χρόνος εκτέλεσης,

MXA_{π} = Μέσος Χρόνος Απόκρισης

XA = Χρόνος Αναμονής,

• FCFS (First Come First Served)

Μέσος Χρόνος Αναμονής:

$$XA(A) = t_2 - t_1 - t_{cpu} = 6 - 0 - 6 = 0ms,$$

$$XA(E) = t_2 - t_1 - t_{cpu} = 19 - 5 - 5 = 9ms$$

$$XA(B) = t_2 - t_1 - t_{cpu} = 10 - 2 - 4 = 4ms,$$

$$XA(Z) = t_2 - t_1 - t_{cpu} = 26 - 6 - 7 = 13ms$$

$$XA(\Gamma) = t_2 - t_1 - t_{cpu} = 11 - 3 - 1 = 7ms,$$

$$MXA = (0 + 4 + 7 + 7 + 9 + 13)/6 = 6,66ms$$

$$XA(\Delta) = t_2 - t_1 - t_{cpu} = 14 - 4 - 3 = 7ms,$$

Μέσος Χρόνος Απόκρισης:

$$XA_{\pi}(A) = t_{ap} - t_1 = 0 - 0 = 0ms,$$

$$XA_{\pi}(E) = t_{ap} - t_1 = 14 - 5 = 9ms,$$

$$XA_{\pi}(B) = t_{ap} - t_1 = 6 - 2 = 4ms,$$

$$XA_{\pi}(Z) = t_{ap} - t_1 = 19 - 6 = 13ms$$

$$XA_{\pi}(\Gamma) = t_{ap} - t_1 = 10 - 3 = 7ms,$$

$$MXA_{\pi} = (0+4+7+7+9+13)/6 = 6,66ms$$

$$XA_{\pi}(\Delta) = t_{ap} - t_1 = 11 - 4 = 7ms,$$

Μέσος Χρόνος Ολοκλήρωσης:

$$XO(A) = t_2 - t_1 = 6 - 0 = 6ms,$$

$$XO(E) = t_2 - t_1 = 19 - 5 = 14ms,$$

$$XO(B) = t_2 - t_1 = 10 - 2 = 8ms,$$

$$XO(A) = t_2 - t_1 = 26 - 6 = 20ms$$

$$XO(\Gamma) = t_2 - t_1 = 11 - 3 = 8ms,$$

$$MXO = (6 + 8 + 8 + 10 + 14 + 20)/6 = 11ms$$

$$XO(\Delta) = t_2 - t_1 = 14 - 4 = 10ms,$$

Πλήθος Θεματικών Εναλλαγών:

$$\#A = 1, \#B = 1, \#\Gamma = 1, \#\Delta = 1, \#E = 1, \#Z = 1$$

• **SJF (Shortest Job First)**

Μέσος Χρόνος Αναμονής:

$$XA(A) = t_2 - t_1 - t_{cpu} = 6 - 0 - 6 = 0ms,$$

$$XA(E) = t_2 - t_1 - t_{cpu} = 19 - 5 - 5 = 9ms,$$

$$XA(\Gamma) = t_2 - t_1 - t_{cpu} = 7 - 3 - 1 = 3ms,$$

$$XA(Z) = t_2 - t_1 - t_{cpu} = 26 - 6 - 7 = 13ms$$

$$XA(\Delta) = t_2 - t_1 - t_{cpu} = 10 - 4 - 3 = 3ms,$$

$$MXA = (0+3+3+8+9+13)/6 = 6ms$$

$$XA(B) = t_2 - t_1 - t_{cpu} = 14 - 2 - 4 = 8ms,$$

Μέσος Χρόνος Απόκρισης:

$$XA\pi(A) = t_{ap} - t_1 = 0 - 0 = 0ms,$$

$$XA\pi(E) = t_{ap} - t_1 = 14 - 5 = 9ms,$$

$$XA\pi(\Gamma) = t_{ap} - t_1 = 6 - 3 = 3ms,$$

$$XA\pi(Z) = t_{ap} - t_1 = 19 - 6 = 13ms$$

$$XA\pi(\Delta) = t_{ap} - t_1 = 7 - 4 = 3ms,$$

$$MXA\pi = (0+3+3+8+9+13)/6 = 6ms$$

$$XA\pi(B) = t_{ap} - t_1 = 10 - 2 = 8ms,$$

Μέσος Χρόνος Ολοκλήρωσης:

$$XO(A) = t_2 - t_1 = 6 - 0 = 6ms,$$

$$XO(E) = t_2 - t_1 = 19 - 5 = 14ms,$$

$$XO(B) = t_2 - t_1 = 14 - 2 = 12ms,$$

$$XO(Z) = t_2 - t_1 = 26 - 6 = 20ms$$

$$XO(\Gamma) = t_2 - t_1 = 7 - 3 = 4ms,$$

$$MXO = (6 + 12 + 4 + 6 + 14 + 20)/6 = 10,33ms$$

$$XO(\Delta) = t_2 - t_1 = 10 - 4 = 6ms,$$

Πλήθος Θεματικών Εναλλαγών:

$$\#A = 1, \#B = 1, \#\Gamma = 1, \#\Delta = 1, \#E = 1, \#Z = 1$$

• **SRTF (Shortest Remaining Time First)**

Μέσος Χρόνος Αναμονής:

$$XA(A) = t_2 - t_1 - t_{cpu} = 7 - 0 - 6 = 1ms,$$

$$XA(E) = t_2 - t_1 - t_{cpu} = 19 - 5 - 5 = 9ms$$

$$XA(B) = t_2 - t_1 - t_{cpu} = 14 - 2 - 4 = 8ms,$$

$$XA(Z) = t_2 - t_1 - t_{cpu} = 26 - 6 - 7 = 13ms$$

$$XA(\Gamma) = t_2 - t_1 - t_{cpu} = 4 - 3 - 1 = 0ms,$$

$$MXA = (1 + 8 + 0 + 3 + 9 + 13)/6 = 5,66ms$$

$$XA(\Delta) = t_2 - t_1 - t_{cpu} = 10 - 4 - 3 = 3ms,$$

Μέσος Χρόνος Απόκρισης:

$$ΧΑπ(A) = tap - t1 = 0 - 0 = 0ms,$$

$$ΧΑπ(B) = tap - t1 = 10 - 2 = 8ms,$$

$$ΧΑπ(Γ) = tap - t1 = 3 - 3 = 0 ms,$$

$$ΧΑπ(Δ) = tap - t1 = 7 - 4 = 3ms,$$

$$ΧΑπ(E) = tap - t1 = 14 - 5 = 9ms,$$

$$ΧΑπ(Z) = tap - t1 = 19 - 6 = 13ms$$

$$ΜΧΑπ = (0 + 8 + 0 + 3 + 9 + 13)/6 = 5,5ms$$

Μέσος Χρόνος Ολοκλήρωσης:

$$ΧΟ(A) = t2 - t1 = 7 - 0 = 7ms,$$

$$ΧΟ(B) = t2 - t1 = 14 - 2 = 12ms,$$

$$ΧΟ(Γ) = t2 - t1 = 4 - 3 = 1ms,$$

$$ΧΟ(Δ) = t2 - t1 = 10 - 4 = 6ms,$$

$$ΧΟ(E) = t2 - t1 = 19 - 5 = 14ms,$$

$$ΧΟ(Z) = t2 - t1 = 26 - 6 = 20ms,$$

$$ΜΧΟ = (7 + 12 + 1 + 6 + 14 + 20)/6 = 10ms$$

Πλήθος Θεματικών Εναλλαγών:

$$\#A = 4(6), \#B = 1, \#Γ = 1, \#Δ = 1, \#E = 1, \#Z = 1$$

• RR (Round Robin) με κβάντο χρόνου 2 χρονικές μονάδες**Μέσος Χρόνος Αναμονής:**

$$ΧΑ(A) = t2 - t1 - tc_{pu} = 17 - 0 - 6 = 11ms,$$

$$ΧΑ(B) = t2 - t1 - tc_{pu} = 11 - 2 - 4 = 5ms,$$

$$ΧΑ(Γ) = t2 - t1 - tc_{pu} = 7 - 3 - 1 = 3ms,$$

$$ΧΑ(Δ) = t2 - t1 - tc_{pu} = 18 - 4 - 3 = 11ms,$$

$$ΧΑ(E) = t2 - t1 - tc_{pu} = 23 - 5 - 5 = 13ms,$$

$$ΧΑ(Z) = t2 - t1 - tc_{pu} = 26 - 6 - 7 = 13ms$$

$$ΜΧΑ = (11 + 5 + 3 + 11 + 13 + 13)/6 = 9,33ms$$

Μέσος Χρόνος Απόκρισης:

$$ΧΑπ(A) = tap - t1 = 0 - 0 = 0ms,$$

$$ΧΑπ(B) = tap - t1 = 2 - 2 = 0ms,$$

$$ΧΑπ(Γ) = tap - t1 = 6 - 3 = 3ms,$$

$$ΧΑπ(Δ) = tap - t1 = 7 - 4 = 3ms,$$

$$ΧΑπ(E) = tap - t1 = 11 - 5 = 6ms,$$

$$ΧΑπ(Z) = tap - t1 = 13 - 6 = 7ms$$

$$ΜΧΑπ = (0 + 0 + 3 + 3 + 6 + 7)/6 = 3,16ms$$

Μέσος Χρόνος Ολοκλήρωσης:

$$XO(A) = t_2 - t_1 = 17 - 0 = 17\text{ms},$$

$$XO(E) = t_2 - t_1 = 23 - 5 = 18\text{ms},$$

$$XO(B) = t_2 - t_1 = 11 - 2 = 9\text{ms},$$

$$XO(Z) = t_2 - t_1 = 26 - 6 = 20\text{ms}$$

$$XO(\Gamma) = t_2 - t_1 = 7 - 3 = 4\text{ms},$$

$$MXO = (17+9+4+14+18+20)/6 = 13,66\text{ms}$$

$$XO(\Delta) = t_2 - t_1 = 18 - 4 = 14\text{ms},$$

Πλήθος Θεματικών Εναλλαγών:

$$\#A = 3, \#B = 2, \#\Gamma = 1, \#\Delta = 2, \#E = 3, \#Z = 4$$

γ)

• Διάγραμμα Gantt

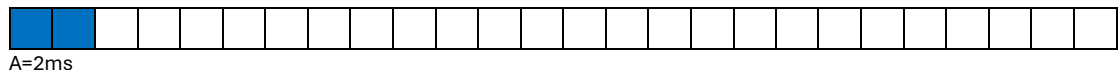
Στον αλγόριθμο LRTFP υλοποιείται κάθε φορά η διεργασία που έχει τον περισσότερο εναπομείναντα χρόνο. Για αυτόν τον λόγο, θα πρέπει μετά από κάθε χρονική μονάδα, δηλαδή μετά από κάθε ms, να σταματάμε για να ελέγχουμε ποιο είναι αυτό που πληρεί το κριτήριο στην συγκεκριμένη περίπτωση.

Τα χρώματα που έχει η κάθε διεργασία



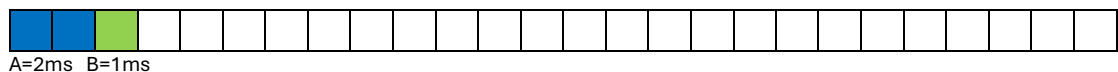
Την χρονική στιγμή 0, φτάνει στον επεξεργαστή η διεργασία A. Την αφήνουμε να τρέξει για 1ms και έπειτα τρέχει για ακόμα 1ms αφού δεν έχει φτάσει κάποια άλλη διεργασία για σύγκριση και έχει διάρκεια εκτέλεσης που μπορεί να εκτελεστεί ξανά.

Άρα τρέχει το A για 2ms και $A = 6 - 2 = 4$:



Την χρονική στιγμή 2 φτάνει στον επεξεργαστή η διεργασία B. Που σε αυτή την περίπτωση αφού έχουμε 2 διεργασίες θα πρέπει να τις συγκρίνουμε.

Βλέπουμε λοιπόν ότι η A και B έχουν το ίδιο εναπομείναντα χρόνο εκτέλεσης. Για αυτόν τον λόγο θα κοιτάξουμε τα PID. Αυτή που έχει το μικρότερο θα εισέλθει πρώτη. Δηλαδή στην συγκεκριμένη περίπτωση η διεργασία B. Όπως είπαμε και προηγουμένως, θα τρέξει μόνο για 1ms και μετά θα σταματήσει για να επαναληφθεί ο έλεγχος. Η B λοιπόν σταματάει την χρονική στιγμή 3. Που τώρα έχουμε για την $B = 4 - 1 = 3$ εναπομείναντα χρόνο εκτέλεσης.

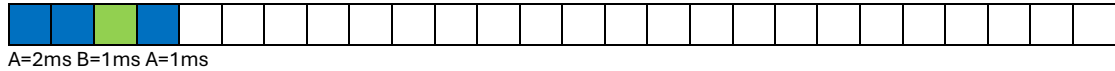


Την χρονική στιγμή φτάνει η διεργασία Γ, η οποία έχει εναπομείναντα χρόνο εκτέλεσης =1.

Άρα αυτή την στιγμή έχουμε:

$A=4$, $B=3$ και $\Gamma=1$ εναπομείναντα χρόνο εκτέλεσης.

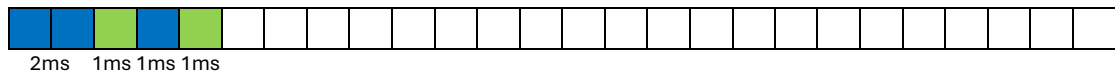
Συγκρίνουμε τον χρόνο που απομένει στην κάθε διεργασία που έχουμε. Αυτή με τον μεγαλύτερο είναι η A, άρα αυτή θα είναι αυτή που θα εκτελεστεί. Εκτελείται για 1ms και έπειτα σταματάει.



Άρα έχουμε $A=4-1=3$

Την χρονική στιγμή 4 φτάνει και η διεργασία Δ με εναπομείναντα χρόνο εκτέλεσης 3.

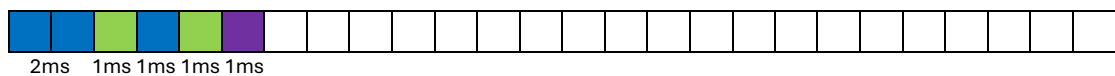
Έχουμε την A, B και Δ που έχουν το ίδιο και μεγαλύτερο εναπομείναντα χρόνο εκτέλεσης, άρα θα πρέπει να κοιτάξουμε στα PIDs. Μικρότερο έχει το B και εκτελείται για 1ms:



Άρα $B=3-1=2$.

Έπειτα την χρονική στιγμή 5 φτάνει η διεργασία Ε έχει εναπομείναντα χρόνο εκτέλεσης 5.

Άρα έχουμε $A=3$ $B=2$ $\Gamma=1$ $\Delta=3$ $E=5$. Αφού η Ε έχει το μεγαλύτερο χρόνο εκτελείται:

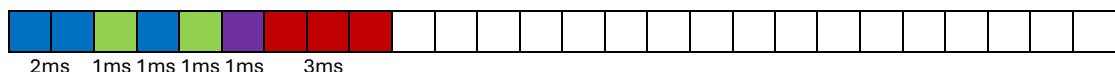


Άρα $E=5-1=4$.

Την χρονική στιγμή 6 φτάνει η Ζ που έχει διάρκεια εκτέλεσης 7.

Άρα έχουμε $A=3$ $B=2$ $\Gamma=1$ $\Delta=3$ $E=4$ $Z=7$.

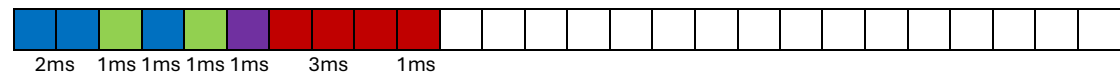
Συγκρίνουμε τον χρόνο που απομένει στην κάθε διεργασία που έχουμε. Αυτή με τον μεγαλύτερο είναι η Ζ, άρα αυτή θα είναι αυτή που θα εκτελεστεί. Εκτελείται για 1ms και έπειτα σταματάει. Ξαναγίνεται έλεγχος και παρατηρούμε πως η Ζ εξακολουθεί να είναι αυτή με τον μεγαλύτερο εναπομείναντα χρόνο. Αυτό επαναλαμβάνεται άλλη μια φορά. Μέχρι που καταλήγουμε να έχουμε 2 διεργασίες που έχουν ισοπαλία στον μεγαλύτερο εναπομείναντα χρόνο



Τώρα που εκτελέσθηκε η Ζ, και έχουμε ισοπαλία των διεργασιών Ε και Ζ αφού είναι σε κατάσταση:

$A=3$ $B=2$ $\Gamma=1$ $\Delta=3$ $E=4$ και $Z=4$.

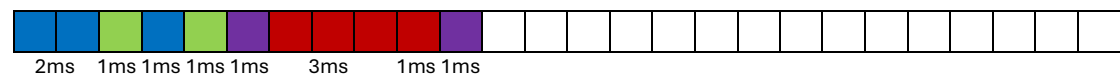
Βλέπουμε τα PIDs των διεργασιών και παρατηρούμε ότι η Z έχει μικρότερο PID. Άρα εκτελείται:



Άρα τώρα έχουμε:

A=3 B=2 Γ=1 Δ=3 E=4 και Z=3.

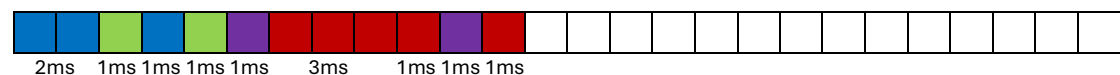
Εκτελείται η E:



Τώρα έχουμε:

A=3 B=2 Γ=1 Δ=3 E=3 και Z=3.

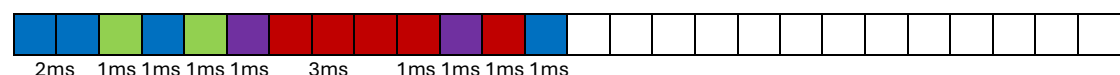
Ισοπαλία των διεργασιών A, Δ, E και Z. Βλέπουμε PIDs και η Z έχει το μικρότερο, εκτελείται:



Άρα έχουμε:

A=3 B=2 Γ=1 Δ=3 E=3 και Z=2.

Έχουμε ισοπαλία των A, Δ και E. Βλέπουμε PIDs και η A έχει το μικρότερο και εκτελούμε:



Άρα έχουμε:

A=2 B=2 Γ=1 Δ=3 E=3 και Z=2.

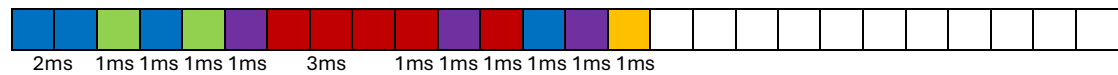
Έχουμε ισοπαλία για Δ και E που έχουν το μεγαλύτερο εναπομείναντα χρόνο άρα βλέπου PIDs ξανά, μικρότερο PID έχει η E εκτελείται:



Τώρα έχουμε:

A=2 B=2 Γ=1 Δ=3 E=2 και Z=2.

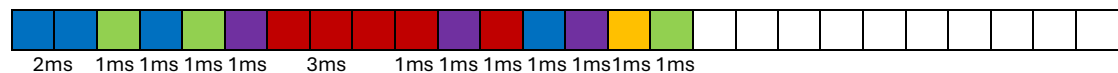
Εκτελείται η Δ:



Στο επόμενο βήμα έχουμε:

A=2 B=2 Γ=1 Δ=2 E=2 και Z=2.

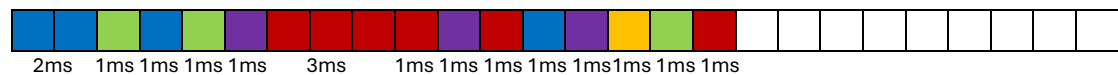
Ισοπαλία των A, B, Δ, E και Z, βλέπουμε τα PIDs εκτελούμε την B αφού έχει το μικρότερο PID. Παρόλο που και η Z έχει το μικρότερο μαζί με την B βλέπουμε το χρόνο άφιξης, δηλαδή αυτή που έχει έρθει πιο πριν. Με αποτέλεσμα να εκτελούμε την B:



Τώρα έχουμε:

A=2 B=1 Γ=1 Δ=2 E=2 και Z=2.

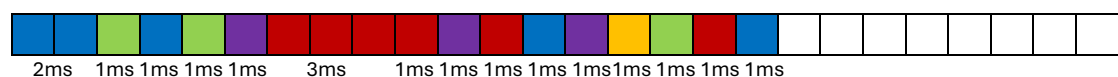
Ισοπαλία των A, Δ, E και Z. Μικρότερο PID έχει η Z και εκτελείται.



Έχουμε:

A=2 B=1 Γ=1 Δ=2 E=2 και Z=1.

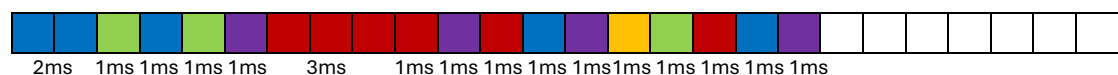
Ισοπαλία των A, Δ και E. Μικρότερο PID έχει το A και εκτελείται.



Έχουμε:

A=1 B=1 Γ=1 Δ=2 E=2 και Z=1.

Ισοπαλία των Δ και E. Μικρότερο PID η E εκτελείται.



Έχουμε:

A=1 B=1 Γ=1 Δ=2 E=1 και Z=1.

Εκτελείται η Δ.

#A = 5, #B = 4, #Γ = 1, #Δ = 3, #E = 5, #Z = 7.